

Day 15 — Linux Fundamentals

Filesystem, Permissions, grep/find & Processes — Full Detail

Why this matters: almost every real data pipeline eventually runs on a Linux server (cloud VMs, Docker containers, Airflow workers all run Linux). These commands are not optional background knowledge - they're used daily.

1. Filesystem Navigation: ls, cd, pwd

Command	What It Does
pwd	Print working directory - shows exactly where you currently are
ls	List files and folders in the current directory
ls -la	List all files (including hidden ones starting with .), with details (permissions, size, date)
cd foldername	Change directory into 'foldername'
cd ..	Move up one directory level
cd ~	Jump straight to your home directory

```
pwd
# /home/akash/projects

ls -la
# drwxr-xr-x  2 akash akash 4096 Jan 10 09:00 data
# -rw-r--r--  1 akash akash  512 Jan 10 09:01 pipeline.py

cd data
cd ..
cd ~
```

Key takeaway: `pwd` and `ls -la` *should be your first two commands whenever you land in an unfamiliar directory - orient yourself before doing anything else.*

2. File Operations: cp, mv, rm

Command	What It Does
cp source dest	Copy a file (source stays in place)
cp -r source dest	Copy a folder and everything inside it (recursive)
mv source dest	Move a file, or rename it if dest is in the same folder

rm filename	Delete a file - permanently, no recycle bin
rm -r foldername	Delete a folder and everything inside it - permanently

```
cp data.csv backup_data.csv
cp -r project_folder project_backup

mv old_name.csv new_name.csv      # renaming
mv data.csv archive/data.csv      # moving into a folder

rm temp_file.txt
rm -r old_logs/
```

Key takeaway: *There is no undo for rm. Before running rm -r on a folder, run ls on it first to double-check exactly what you're about to delete.*

3. Viewing Files: cat, less, head, tail

Command	What It Does
cat file	Print the entire file to the screen at once
less file	Open the file for scrollable viewing (press q to quit) - better for large files
head file	Show the first 10 lines of a file
head -n 20 file	Show the first 20 lines
tail file	Show the last 10 lines of a file
tail -f logfile	Follow a file live - keeps printing new lines as they're written (essential for watching a running pipeline's log)

```
cat config.py
less pipeline.log      # scroll through a big log file

head -n 5 data.csv
# peek at the first 5 rows without opening the whole file

tail -f pipeline.log
# watch a pipeline's log update live while it runs
```

Key takeaway: *tail -f is one of the most-used commands in real data engineering work - it's how you watch a pipeline's log output in real time while it's actively running on a server.*

4. Searching: grep and find

grep - search for text **INSIDE** files:

```
grep "ERROR" pipeline.log
# find lines containing "ERROR"

grep -i "error" pipeline.log
# case-insensitive search

grep -r "TODO" ./src
# search recursively through all files in a folder

grep -c "ERROR" pipeline.log
# count how many matching lines exist
```

find - search for files/folders BY NAME or attribute:

```
find . -name "*.csv"
# find all CSV files from the current folder down

find . -name "*.py" -newer script.py
# find Python files modified after script.py

find . -size +10M
# find files larger than 10 MB

find . -type d -name "logs"
# find folders (not files) named "logs"
```

The distinction that matters: grep looks *inside* file contents for matching text; find looks at file/folder *names and attributes* (size, date, type) without caring what's inside them.

Key takeaway: A very common real debugging pattern: `grep "ERROR" pipeline.log` *to instantly find every failure in a log file that might be thousands of lines long.*

5. Pipes & Redirection

Pipe (|) - sends one command's output as the next command's input:

```
cat pipeline.log | grep "ERROR"
# cat prints the file; grep filters to just error lines

cat pipeline.log | grep "ERROR" | wc -l
# wc -l (word count, lines mode) counts the error lines
```

Redirection - sends output to a file instead of the screen:

Operator	Behavior
>	Write output to a file, overwriting anything already there
>>	Append output to the end of a file, keeping existing content
<	Use a file as input to a command

```
python pipeline.py > output.log
# overwrite output.log with this run's output

python pipeline.py >> output.log
# append this run's output to the existing log

sort < names.txt
# use names.txt as input to the sort command
```

Key takeaway: Pipes let you chain small, simple commands into a powerful one-liner - this is a core Unix philosophy: each tool does one thing well, and you combine them rather than needing one giant command that does everything.

6. Permissions

Every file has permissions controlling who can read, write, or execute it, shown by `ls -la` as a 10-character string like `-rwxr-xr--`.

```
-rwxr-xr--

-      : file type (- = regular file, d = directory)
rwx   : Owner permissions  -> read, write, execute
r-x   : Group permissions  -> read, execute (no write)
r--   : Others permissions -> read only
```

Changing permissions with `chmod`:

```
chmod +x pipeline.sh
# make a script executable

chmod 644 config.py
# owner: read/write, group/others: read only

chmod 755 run_pipeline.sh
# owner: read/write/execute, group/others: read/execute
```

The numbers explained: each digit represents read(4)+write(2)+execute(1) added together for owner/group/others. 7 = 4+2+1 (all three), 5 = 4+1 (read+execute, no write).

Key takeaway: A shell script must have execute permission (`chmod +x`) before you can run it directly - this is one of the most common "permission denied" errors beginners hit.

7. Processes

Command	What It Does
ps	List currently running processes for your session
ps aux	List ALL running processes on the system, with detail (user, CPU, memory)

top	Live, auto-refreshing view of running processes and resource usage
kill PID	Stop a process by its Process ID
kill -9 PID	Force-kill a process that won't stop normally
command &	Run a command in the background, freeing up your terminal

```
ps aux | grep python
# find all running Python processes

python pipeline.py &
# run in the background, returns terminal control immediately
# [1] 12345  <- this number is the PID

kill 12345
# stop that background process
```

Key takeaway: When a pipeline seems stuck or is consuming too much memory, `ps aux` or `top` is how you find its process ID, and `kill` is how you stop it.

Day 15 Summary — Linux Fundamentals

Category	One-Line Recap
Navigation	<code>pwd</code> , <code>ls</code> , <code>cd</code> - always orient yourself first
File Operations	<code>cp</code> , <code>mv</code> , <code>rm</code> - <code>rm</code> has no undo, double-check before running
Viewing Files	<code>cat/less</code> for full view, <code>head/tail</code> for a peek, <code>tail -f</code> to watch live
Searching	<code>grep</code> searches inside file contents; <code>find</code> searches file names/attributes
Pipes & Redirection	<code> </code> chains commands; <code>></code> overwrites a file, <code>>></code> appends to it
Permissions	<code>rwx</code> per owner/group/others; <code>chmod +x</code> needed to run a script
Processes	<code>ps/top</code> to see what's running; <code>kill</code> to stop it; <code>&</code> to run in the background

Overall takeaway: These commands are the baseline literacy for working on any server, container, or cloud VM - every tool from Day 35 (Airflow) onward assumes you can navigate a Linux filesystem, read a log file, and manage a running process without help.